ТЕМА 3. КАСКАДНЫЕ ТАБЛИЦЫ СТИЛЕЙ

ЛЕКЦИЯ 3.52. ФОРМЫ.

Введение. Настройки элементов по умолчанию. Простые стили. Стилизация простых элементов. Стилизация сложных элементов.

Введение

Начнем с того, что никаких специальных стилей у элементов формы нет. Возникает вопрос, зачем мы тогда выносим их стилизацию в отдельное занятие? А потому что не все с ними так просто.

Исторически сложилось, что стилизации элементы управления в форме подвергаются из рук вон плохо. И если настроить внешний вид поля ввода текста еще можно, то вот управлять значком радиокнопки или чекбокса очень и очень тяжело.

Вообще все элементы формы условно делят на три группы:

Легко стилизуются:

1. **form.**
2. **fieldset** и **legend.**
3. Однострочный текст **input** (например, тип текста, URL, электронная почта), за исключением <input type="search">.
4. Многострочный **textarea**.
5. Кнопки (**input** и **button**).
6. **label**.
7. **output**.

Сложно стилизуются

1. Флажки и радиокнопки.
2. <input type="search">.

©Pechora-PRO

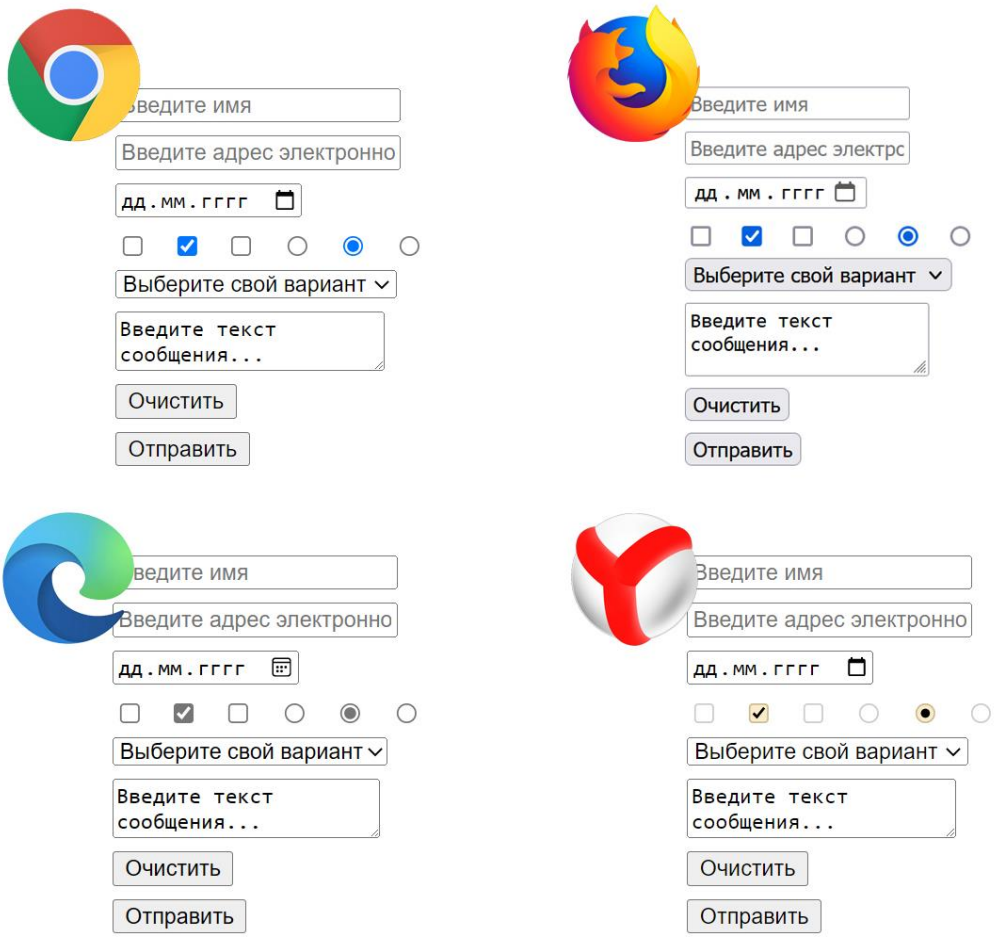
Нельзя стилизовать только средствами CSS

1. <input type="color">.
2. Элементы управления, связанные с датой.
3. <input type="range">.
4. <input type="file">.
5. Элементы, участвующие в создании выпадающих виджетов, в том числе select, option, optgroup, datalist.
6. progress и meter.

Важно! Сейчас есть псевдоэлементы для настройки внешнего вида этих элементов, но они не согласованы между браузерами, поэтому не очень надежны.

Начинаем с общих подходов к стилизации, а продолжим занятие уроком как создать собственные уникальные элементы управления формой.

Настройки элементов по умолчанию



©Pechora-PRO

Все элементы разметки имеют базовый набор стилей, который в достаточной степени зависит от браузера, в котором мы страницу просматриваем. Нашей задачей при настройке элементов станет эти стили переопределить.

Еще часть настроек приходит к элементам формы от операционной системы. Мы можем запретить элементам наследовать их с помощью свойства **appearance** (см. подробности в задании к лекции).

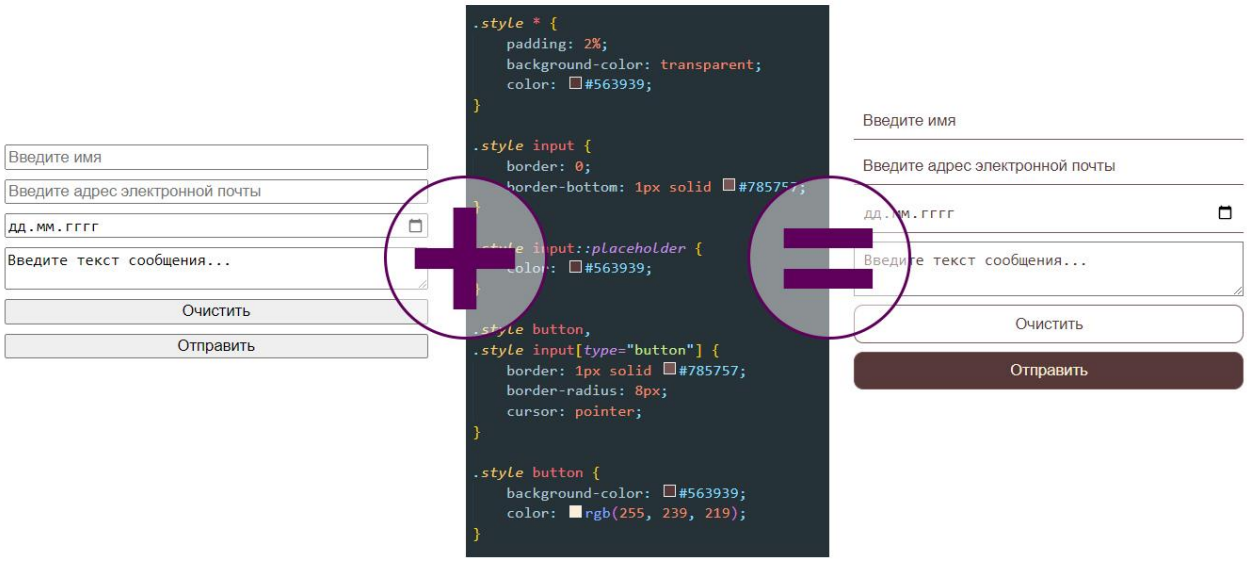
Простые стили

Среди свойств, которые чаще всего используют для простой стилизации можно выделить:

- Padding (внутренние отступы)
- Margin (внешние отступы)
- Border (граница)
- Box shadow (тень блока)
- Border radius (для скругления границ)
- Width (ширина)
- Font (шрифт)

Но при их использовании есть некоторые нюансы.

©Pechora-PRO



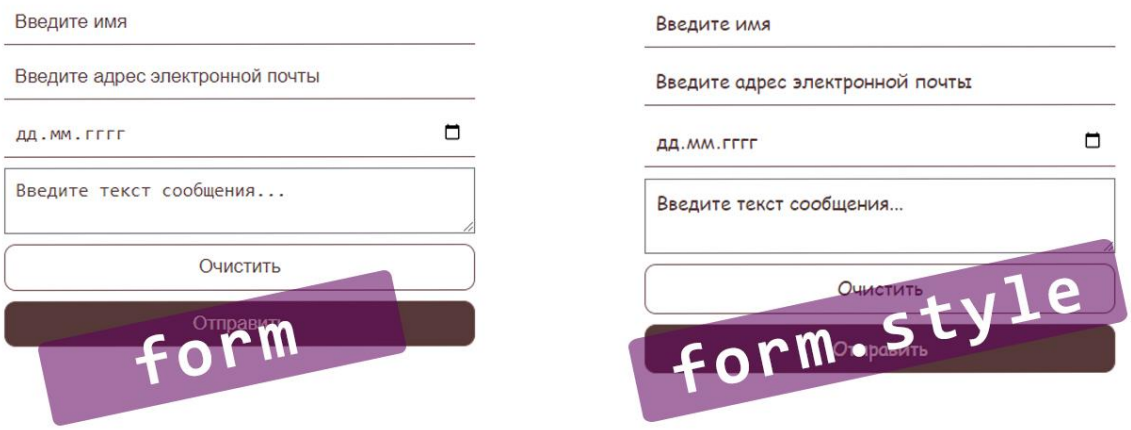
Шрифты и текст

По умолчанию некоторые элементы формы не наследуют **font-family** и **font-size** от своих родителей. Вместо этого многие браузеры используют внешний вид системы по умолчанию. Чтобы стиль ваших форм соответствовал остальному содержимому, вы можете добавить для них настройку стиля со значением **inherit**. Например,

©Pechora-PRO

```css
body {
    font-family: 'Comic Sans MS', Arial, sans-serif;
}

.style button,
.style input,
.style select,
.style textarea {
    font-family: inherit;
}
```



Значение **inherit** заставляет значение свойства совпадать с вычисленным значением свойства его родительского элемента.

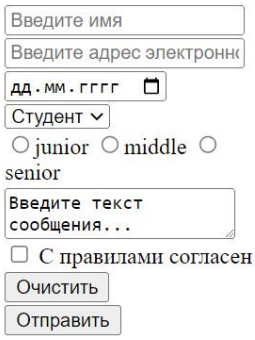
Настройки блока

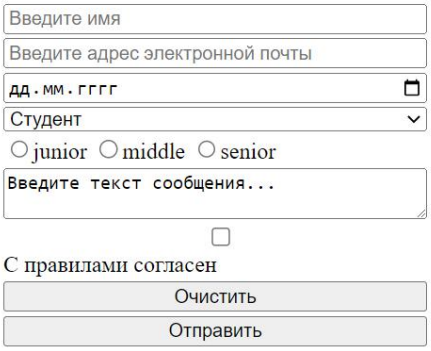
Все текстовые поля имеют полную поддержку всех свойств, связанных с блочной моделью CSS, таких как width, height, padding, margin и border. Однако, как и прежде, браузеры полагаются на системные стили по умолчанию при их отображении.

Например, чтобы придать одинаковый размер нескольким различным элементам, нужно использовать свойство **box-sizing**. В своих проектах мы будем применять это свойство ко всем элементам страницы через селекторы *, **::before**, **::after**.

©Pechora-PRO

```
*{box-sizing:border-box;}
```





`width:auto` `width:300px`

Размещение легенды

Элемент **legend** можно стилизовать, но может быть сложно расположить его в месте отличном от верхнего левого угла (по умолчанию).

Самый простой вариант – использовать абсолютное позиционирование относительно родителя **fieldset**.

```
<style>
    fieldset {
        position: relative;
    }

    legend {
        position: absolute;
        bottom: 0;
        right: 0;
    }
</style>
```

Размещаем легенду в правом нижнем углу группы.



Элемент **legend** очень важен для доступности вашей страницы — вспомогательные технологии будут произносить его как часть метки каждого элемента формы.

Важно. Вы также можете использовать свойство **transform**. О нем в следующей теме.

Текстовые области

©Pechora-PRO

Элементы **textarea** по умолчанию отображаются как встроенные блочные элементы. Для их настройки кроме указания на количество строк и символов важно знать и применять свойства **resize** и **overflow**.

Свойство **overflow** задает желаемое поведение, когда содержимое не помещается в поле родительского элемента (переполняется) в горизонтальном и/или вертикальном направлении. Можно развернуть в два свойства **overflow-x** для горизонтали и **overflow-y** для вертикали. Чтобы свойство сработало, блок должен иметь заданную высоту (**height** или **max-height**), либо **white-space** со значением **nowrap**.

Возможные значения:

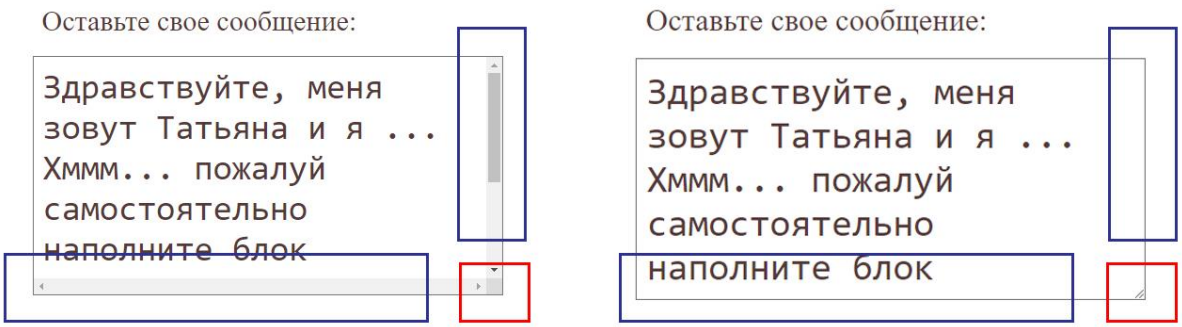
- **visible -** содержимое при переполнении не обрезается и видимо за пределами поля заполнения элемента. Поле элемента не является контейнером прокрутки. Это значение по умолчанию.
- **hidden -** содержимое переполнения обрезается в поле заполнения элемента. Полосы прокрутки отсутствуют, а обрезанное содержимое не скрыто, но все еще существует. При этом содержимое можно прокручивать программно. Поле элемента представляет собой контейнер прокрутки.
- **clip** - содержимое переполнения обрезается по краю переполнения элемента, который определяется с помощью свойства **overflow-clip-margin**. В результате содержимое переполняет поле заполнения элемента на значение **overflow-clip-margin** или на **0px** если оно не установлено. Поле не имеет прокрутки.
- **scroll -** содержимое переполнения обрезается в поле заполнения элемента, а содержимое переполнения можно прокручивать с помощью полос прокрутки. Браузеры отображают полосы прокрутки, как в горизонтальном, так и в вертикальном направлениях, если установлено только

©Pechora-PRO

одно значение, независимо от того, переполняется или обрезается какой-либо контент. Таким образом, использование этого ключевого слова может предотвратить появление и исчезновение полос прокрутки при изменении содержимого. При печати контент переполнения может быть напечатан.

- **auto** – аналогично **scroll**, но браузеры отображают полосы прокрутки только в том случае, если содержимое переполняется, и по умолчанию их скрывают.

```
overflow:scroll; overflow:hidden;
```




```
resize:none;        resize:both;
```

Свойство **resize** настраивает возможность изменять размер нашего многострочного текстового поля, да и любого другого элемента. **Resize** не работает со встроенными элементами и блочными, у которых свойству **overflow** установлено значение **visible**.

Возможные значения:

- **none** – размер изменить нельзя.
- **both** – меняем размер и по горизонтали и по вертикали.
- **horizontal** – меняем размер только по горизонтали.
- **vertical** – меняем размер только по вертикали.

Селектор по типу атрибута

©Pechora-PRO

Если для стилизации обычных элементов страницы порой достаточно классов и идентификаторов, то при работе с формой, помощником для нас станет селектор атрибутов.

Вспоминаем синтаксис селектора (полная версия в уроке 3.5 Селекторы CSS)

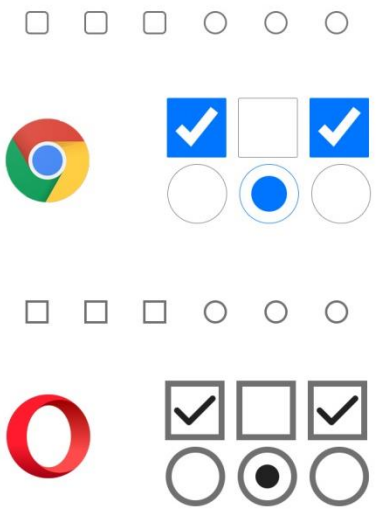
Синтаксис	Пример	Описание

[ИмяАтрибута]	[required] {
	…
	}
		селектор **[required]** выберет все элементы, у которых есть атрибут **required** , т.е. все элементы обязательные для заполнения

НазваниеЭлемента . [ИмяАтрибута]	input[required] {
	…
	}
		выберет из всех элементов с атрибутом **required** только **input**

[ИмяАтрибута =Значение]	input[type=tel] {
	…
	}
		стилизует элемент если значение атрибута полностью совпадает с указанным. Например, только поля для ввода телефона.

Стилизация сложных элементов



Размеры флажков и радиокнопок не предназначены для изменения их дизайна по умолчанию, и браузеры реагируют совершенно по-разному, когда вы пытаетесь это сделать.

Например, при изменении ширины и высоты один браузер увеличит только размеры области, второй пропорционально увеличит и границу элемента.

©Pechora-PRO

При этом установка цвета для содержимого, фона или границы не даст никакого эффекта.

Стилизация checkbox-а будет для нас состоять из нескольких этапов:

- удаление старых настроек;
- установка новых настроек размера, границы и цвета содержимого;
- добавление нового изображения «галочки» в псевдоэлемент;
- настройка показа «галочки» при выборе checkbox-а;
- добавление стиля для неактивного элемента.

```css
/* Шаг 1  Сбрасываем настройки по умолчанию*/
input[type="checkbox"] {
    appearance: none;
}

/* Шаг 2  Задаем новые настройки*/
input[type="checkbox"] {
    position: relative;
    width: 150px;
    height: 150px;
    border: 3px solid rgb(86, 9, 99);
    border-radius: 8px;
    color: rgb(255, 0, 221);
}
```

```css
/* Шаг 3 Добавляем новую галочку перед элементом*/

input[type="checkbox"]::before {
    content: "✓";
    position: absolute;
    visibility: hidden;
    font-size: 130px;
    left: 20px;
    top: -20px;
}

/* Шаг 4 Настраиваем показ при выборе checkbox-а*/

input[type="checkbox"]:checked::before {
    visibility: visible;
}
```

```css
/* Шаг 5 Настраиваем checkbox в неактивном состоянии*/

input[type="checkbox"]:disabled {
    border-color: black;
    background: #ddd;
    color: gray;
}
```



Аналогично можно настроить и внешний вид радиокнопок.



normal|checked|disabled

Еще один вариант, воспользоваться связью между элементами **label** и **input** через атрибут **for**.

Если у вас есть метка, связанная с элементом управления формы, щелчок по метке активирует элемент управления. А

©Pechora-PRO

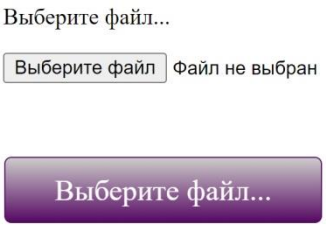
значит можно скрыть элемент, а метку стилизовать как вам угодно.

```html
<div class="container">
    <div>
        <form>
            <label for="sample">Выберите файл...</label>
            <input type="file" name="sample" id="sample">
        </form>
    </div>
</div>
```

```css
<style>

    input[type="file"] { display: none; }
    label[for="sample"] {
        display: block;
        width: 200px;
        padding: 2%;
        border: 1px solid #560963;
        border-radius: 5px;
        background: linear-gradient(to bottom, #cecccc, #560963);
        text-align: center;
        color: #fff;
        font-size: 1.3em;
    }

    label[for="sample"]:hover {
        background: linear-gradient(to bottom, #fff, #8a1b9d);
    }
</style>
```



К сожалению, для некоторых элементов часть их внутреннего содержания буквально невозможно стилизовать.

Например, все типы ввода даты/времени (datetime-local, time, week, month) имеют одну и ту же основную проблему. Внутренние части элемента управления (всплывающий календарь для выбора даты, счетчик для увеличения/уменьшения значений) вообще не поддаются стилю, и вы не сможете избавиться от них даже с помощью appearance: none;. При этом обертку так же легко стилизовать, как и любое поле для ввода текста.

Поэтому остается или смириться с различиями при просмотре страниц в разных браузерах или создавать свои собственные элементы и библиотеки, в том числе с использованием JavaScript.

©Pechora-PRO